Terminal Bot Integration

Telegram & WhatsApp Command Execution

Date: March 13, 2026
Purpose: Send terminal commands via Telegram/WhatsApp and receive real-time output
Status: Ready to Deploy

OPTION 1: TELEGRAM BOT (RECOMMENDED)

Step 1: Create Telegram Bot

Step 1.1: Open Telegram and Create Bot

```
1. Open Telegram app or web (web.telegram.org)
2. Search for "@BotFather"
3. Send: /start
4. Send: /newbot
5. Follow prompts:
   - Bot name: "AGLC CRM Terminal"
   - Bot username: "aglc_crm_terminal_bot" (must be unique)
6. Copy the bot token (looks like: 123456:ABC-DEF1234ghIkl-
zyx57W2v1u123ew11)
```

Step 1.2: Get Your Chat ID

```
1. Send a message to your new bot
2. Open this URL in browser:
   https://api.telegram.org/bot<YOUR_BOT_TOKEN>/getUpdates
3. Find your chat_id in the response
4. Save both TOKEN and CHAT_ID
```

Step 2: Install Telegram Bot Framework

```
# Install Node.js Telegram bot library
cd ~/projects/aglc
mkdir telegram-bot
cd telegram-bot

# Initialize project
npm init -y

# Install dependencies
npm install telegraf dotenv child_process

# Create .env file
cat > .env << 'EOF'
TELEGRAM_BOT_TOKEN=YOUR_BOT_TOKEN_HERE
TELEGRAM_CHAT_ID=YOUR_CHAT_ID_HERE
EOF
```

Step 3: Create Telegram Bot Script

```
cat > ~/projects/aglc/telegram-bot/bot.js << 'EOF'
const { Telegraf } = require('telegraf');
const { exec } = require('child_process');
const fs = require('fs');
const path = require('path');
require('dotenv').config();

const bot = new Telegraf(process.env.TELEGRAM_BOT_TOKEN);
const allowedChatId = parseInt(process.env.TELEGRAM_CHAT_ID);

// Store active processes
const activeProcesses = {};

// Log function
function log(message) {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${message}`);
}

// Execute command and send output
function executeCommand(ctx, command) {
  return new Promise((resolve) => {
    log(`Executing: ${command}`);

    const process = exec(command, { cwd: process.env.HOME, maxBuffer: 1024 *
1024 * 10 });
    let output = '';
    let errorOutput = '';

    process.stdout.on('data', (data) => {
      output += data.toString();
      log(`Output: ${data.toString().substring(0, 100)}`);
    });

    process.stderr.on('data', (data) => {
      errorOutput += data.toString();
      log(`Error: ${data.toString().substring(0, 100)}`);
    });

    process.on('close', (code) => {
      log(`Command completed with code: ${code}`);

      const result = {
```

```
        command,
        exitCode: code,
        output: output || '(no output)',
        error: errorOutput || '(no errors)',
        timestamp: new Date().toISOString()
      };

      // Format response
      let response = `✅ Command Executed\n\n`;
      response += `📝 Command: \`${command}\`\n`;
      response += `📊 Exit Code: ${code}\n`;
      response += `⏰ Time: ${result.timestamp}\n\n`;

      if (output) {
        response += `📤 Output:\n\`\`\`\n${output.substring(0,
3000)}\n\`\`\`\n`;
      }

      if (errorOutput) {
        response += `⚠️ Errors:\n\`\`\`\n${errorOutput.substring(0,
3000)}\n\`\`\`\n`;
      }

      if (output.length > 3000 || errorOutput.length > 3000) {
        response += `\n📌 Output truncated (too long)`;
      }

      // Send response
      ctx.reply(response, { parse_mode: 'Markdown' });
      resolve(result);
    });
  });
}

// Start command
bot.command('start', (ctx) => {
  if (ctx.chat.id !== allowedChatId) {
    ctx.reply('❌ Unauthorized access');
    return;
  }

  ctx.reply(`
🤖 AGLC CRM Terminal Bot

Available commands:
/cmd <command> - Execute terminal command
```

```
/deploy - Run deployment script
/status - Check system status
/docker - Docker commands
/help - Show this help message
/stop - Stop the bot

Example:
/cmd ls -la
/cmd docker-compose ps
/cmd pnpm dev
  `, { parse_mode: 'Markdown' });
});

// Command execution
bot.command('cmd', (ctx) => {
  if (ctx.chat.id !== allowedChatId) {
    ctx.reply('❌ Unauthorized');
    return;
  }

  const command = ctx.message.text.replace('/cmd ', '').trim();

  if (!command) {
    ctx.reply('❌ Please provide a command\nUsage: /cmd <command>');
    return;
  }

  ctx.reply(`⏳ Executing: \`${command}\``, { parse_mode: 'Markdown' });
  executeCommand(ctx, command);
});

// Deployment command
bot.command('deploy', (ctx) => {
  if (ctx.chat.id !== allowedChatId) {
    ctx.reply('❌ Unauthorized');
    return;
  }

  ctx.reply('⏳ Starting deployment...');
  executeCommand(ctx, 'bash ~/AGLC_CRM_DEPLOY.sh');
});

// Status command
bot.command('status', (ctx) => {
  if (ctx.chat.id !== allowedChatId) {
    ctx.reply('❌ Unauthorized');
```

```
    return;
  }

  ctx.reply('⏳ Checking system status...');
  executeCommand(ctx, 'bash ~/verify_installation.sh');
});

// Docker commands
bot.command('docker', (ctx) => {
  if (ctx.chat.id !== allowedChatId) {
    ctx.reply('❌ Unauthorized');
    return;
  }

  const subcommand = ctx.message.text.replace('/docker ', '').trim() ||
'ps';

  const commands = {
    'ps': 'docker-compose ps',
    'logs': 'docker-compose logs -f postgres',
    'up': 'cd ~/projects/aglc/aglc-crm && docker-compose up -d',
    'down': 'cd ~/projects/aglc/aglc-crm && docker-compose down',
    'restart': 'cd ~/projects/aglc/aglc-crm && docker-compose restart'
  };

  const command = commands[subcommand] || `docker-compose ${subcommand}`;
  ctx.reply(`⏳ Running: \`${command}\``, { parse_mode: 'Markdown' });
  executeCommand(ctx, command);
});

// Help command
bot.command('help', (ctx) => {
  if (ctx.chat.id !== allowedChatId) {
    ctx.reply('❌ Unauthorized');
    return;
  }

  ctx.reply(`
📚 AGLC CRM Terminal Bot - Commands

🔧 Basic Commands:
/cmd <command> - Execute any terminal command
/status - Check deployment status
/help - Show this help

🚀 Deployment Commands:
```

```
/deploy - Run full deployment
/docker ps - Show Docker containers
/docker up - Start containers
/docker down - Stop containers
/docker logs - View PostgreSQL logs

📋 Examples:
/cmd pnpm dev
/cmd docker ps
/cmd pnpm test
/cmd git status
/cmd npm list

⚙️ System Info:
/cmd uname -a
/cmd df -h
/cmd free -h

🛑 Stop Bot:
/stop - Stop the bot

  `, { parse_mode: 'Markdown' });
});

// Stop command
bot.command('stop', (ctx) => {
  if (ctx.chat.id !== allowedChatId) {
    ctx.reply('❌ Unauthorized');
    return;
  }

  ctx.reply('🛑 Bot stopping...');
  process.exit(0);
});

// Handle text messages (for quick commands)
bot.on('text', (ctx) => {
  if (ctx.chat.id !== allowedChatId) {
    ctx.reply('❌ Unauthorized');
    return;
  }

  // If message starts with /, it's a command
  if (ctx.message.text.startsWith('/')) {
    return;
  }
```

```
  // Otherwise treat as command
  const command = ctx.message.text;
  ctx.reply(`⏳ Executing: \`${command}\``, { parse_mode: 'Markdown' });
  executeCommand(ctx, command);
});

// Error handling
bot.catch((err, ctx) => {
  log(`Error: ${err}`);
  ctx.reply(`❌ Error: ${err.message}`);
});

// Launch bot
bot.launch();
log('🤖 Telegram bot started');

// Graceful shutdown
process.once('SIGINT', () => bot.stop('SIGINT'));
process.once('SIGTERM', () => bot.stop('SIGTERM'));

EOF

# Install dependencies
npm install
```

Step 4: Start Telegram Bot

```
# Set environment variables
export TELEGRAM_BOT_TOKEN="YOUR_BOT_TOKEN"
export TELEGRAM_CHAT_ID="YOUR_CHAT_ID"

# Start the bot
node ~/projects/aglc/telegram-bot/bot.js

# Or run in background
nohup node ~/projects/aglc/telegram-bot/bot.js > ~/telegram-bot.log 2>&1 &

# To stop the bot
pkill -f "node.*telegram-bot/bot.js"
```

Step 5: Test Telegram Bot

```
In Telegram:
1. Send: /start
2. Send: /status
3. Send: /cmd ls -la
4. Send: /deploy
5. Send: /docker ps
```

OPTION 2: WHATSAPP BOT (USING TWILIO)

Step 1: Setup Twilio Account

```
# 1. Go to https://www.twilio.com/console
# 2. Sign up for free account
# 3. Get Account SID and Auth Token
# 4. Enable WhatsApp Sandbox
# 5. Get WhatsApp number (e.g., +1234567890)
```

Step 2: Create WhatsApp Bot

```
# Create WhatsApp bot directory
cd ~/projects/aglc
mkdir whatsapp-bot
cd whatsapp-bot

# Initialize project
npm init -y

# Install dependencies
npm install twilio express dotenv body-parser

# Create .env file
cat > .env << 'EOF'
TWILIO_ACCOUNT_SID=your_account_sid
TWILIO_AUTH_TOKEN=your_auth_token
TWILIO_WHATSAPP_NUMBER=+1234567890
USER_WHATSAPP_NUMBER=+966XXXXXXXXX
PORT=3000
EOF
```

Step 3: Create WhatsApp Bot Script

```
cat > ~/projects/aglc/whatsapp-bot/server.js << 'EOF'
const express = require('express');
const bodyParser = require('body-parser');
const twilio = require('twilio');
const { exec } = require('child_process');
require('dotenv').config();

const app = express();
app.use(bodyParser.urlencoded({ extended: false }));

const client = twilio(
  process.env.TWILIO_ACCOUNT_SID,
  process.env.TWILIO_AUTH_TOKEN
);

const twilioNumber = process.env.TWILIO_WHATSAPP_NUMBER;
const userNumber = process.env.USER_WHATSAPP_NUMBER;

// Log function
function log(message) {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${message}`);
}

// Execute command
function executeCommand(command) {
  return new Promise((resolve) => {
    log(`Executing: ${command}`);

    exec(command, { cwd: process.env.HOME, maxBuffer: 1024 * 1024 * 10 },
(error, stdout, stderr) => {
      const result = {
        command,
        error: error ? error.message : null,
        stdout: stdout || '(no output)',
        stderr: stderr || '(no errors)'
      };

      log(`Command completed: ${command}`);
      resolve(result);
    });
  });
}
```

```
// Webhook for incoming messages
app.post('/whatsapp', async (req, res) => {
  const incomingMessage = req.body.Body;
  const fromNumber = req.body.From;

  log(`Message from ${fromNumber}: ${incomingMessage}`);

  // Security check
  if (!fromNumber.includes(userNumber.replace('+', ''))) {
    log('Unauthorized access attempt');
    res.status(403).send('Unauthorized');
    return;
  }

  const twiml = new twilio.twiml.MessagingResponse();

  try {
    // Parse commands
    if (incomingMessage.toLowerCase() === '/help') {
      twiml.message(`
🤖 AGLC CRM WhatsApp Bot

Commands:
/status - Check deployment status
/deploy - Run deployment
/docker ps - Show containers
/docker up - Start containers
/docker down - Stop containers
/cmd <command> - Execute command

Examples:
/cmd ls -la
/cmd pnpm dev
/cmd docker ps
      `);
    } else if (incomingMessage.toLowerCase() === '/status') {
      twiml.message('⏳ Checking status...');
      const result = await executeCommand('bash ~/verify_installation.sh');
      twiml.message(`✅ Status:\n${result.stdout.substring(0, 1000)}`);
    } else if (incomingMessage.toLowerCase() === '/deploy') {
      twiml.message('⏳ Starting deployment...');
      const result = await executeCommand('bash ~/AGLC_CRM_DEPLOY.sh');
      twiml.message(`✅ Deployment:\n${result.stdout.substring(0, 1000)}`);
    } else if (incomingMessage.toLowerCase().startsWith('/docker')) {
      const subcommand = incomingMessage.replace('/docker', '').trim() ||
```

```
'ps';
      const commands = {
        'ps': 'docker-compose ps',
        'up': 'cd ~/projects/aglc/aglc-crm && docker-compose up -d',
        'down': 'cd ~/projects/aglc/aglc-crm && docker-compose down',
        'logs': 'docker-compose logs -f postgres'
      };
      const cmd = commands[subcommand] || `docker-compose ${subcommand}`;
      twiml.message(`⏳ Running: ${cmd}`);
      const result = await executeCommand(cmd);
      twiml.message(`✅ Result:\n${result.stdout.substring(0, 1000)}`);
    } else if (incomingMessage.toLowerCase().startsWith('/cmd')) {
      const command = incomingMessage.replace('/cmd', '').trim();
      twiml.message(`⏳ Executing: ${command}`);
      const result = await executeCommand(command);
      twiml.message(`✅ Output:\n${result.stdout.substring(0, 1000)}`);
    } else {
      // Treat as command
      twiml.message(`⏳ Executing: ${incomingMessage}`);
      const result = await executeCommand(incomingMessage);
      twiml.message(`✅ Output:\n${result.stdout.substring(0, 1000)}`);
    }
  } catch (error) {
    log(`Error: ${error.message}`);
    twiml.message(`❌ Error: ${error.message}`);
  }

  res.type('text/xml').send(twiml.toString());
});

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'ok' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  log(`🤖 WhatsApp bot listening on port ${PORT}`);
});

EOF

npm install
```

Step 4: Setup Webhook with Ngrok

```
# Install ngrok
curl https://bin.equinox.io/c/4VmDzA7iaHb/ngrok-stable-linux-amd64.zip -o
ngrok.zip
unzip ngrok.zip
sudo mv ngrok /usr/local/bin/

# Start ngrok tunnel
ngrok http 3000

# Copy the HTTPS URL (e.g., https://abc123.ngrok.io)

# Configure Twilio Webhook:
# 1. Go to Twilio Console
# 2. Messaging > Settings > WhatsApp Sandbox Settings
# 3. When a message comes in: https://abc123.ngrok.io/whatsapp
# 4. Save

# Start WhatsApp bot
node ~/projects/aglc/whatsapp-bot/server.js
```

Step 5: Test WhatsApp Bot

```
In WhatsApp:
1. Send: /help
2. Send: /status
3. Send: /cmd ls -la
4. Send: /deploy
5. Send: /docker ps
```

OPTION 3: SYSTEMD SERVICE (AUTO-START)

Create Systemd Service for Telegram Bot

```
# Create systemd service file
sudo tee /etc/systemd/system/aglc-telegram-bot.service > /dev/null << 'EOF'
[Unit]
Description=AGLC CRM Telegram Bot
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/projects/aglc/telegram-bot
Environment="TELEGRAM_BOT_TOKEN=YOUR_BOT_TOKEN"
Environment="TELEGRAM_CHAT_ID=YOUR_CHAT_ID"
ExecStart=/usr/bin/node /home/ubuntu/projects/aglc/telegram-bot/bot.js
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
EOF

# Enable and start service
sudo systemctl daemon-reload
sudo systemctl enable aglc-telegram-bot
sudo systemctl start aglc-telegram-bot

# Check status
sudo systemctl status aglc-telegram-bot

# View logs
sudo journalctl -u aglc-telegram-bot -f
```

QUICK REFERENCE: TELEGRAM COMMANDS

Command	Purpose	Example
`/start`	Initialize bot	`/start`
`/cmd`	Execute command	`/cmd ls -la`
`/deploy`	Run deployment	`/deploy`
`/status`	Check status	`/status`
`/docker ps`	Show containers	`/docker ps`
`/docker up`	Start containers	`/docker up`
`/docker down`	Stop containers	`/docker down`
`/docker logs`	View logs	`/docker logs`
`/help`	Show help	`/help`
`/stop`	Stop bot	`/stop`

QUICK REFERENCE: WHATSAPP COMMANDS

Command	Purpose	Example
`/help`	Show help	`/help`
`/status`	Check status	`/status`
`/deploy`	Run deployment	`/deploy`
`/cmd`	Execute command	`/cmd ls -la`
`/docker ps`	Show containers	`/docker ps`
`/docker up`	Start containers	`/docker up`
`/docker down`	Stop containers	`/docker down`

SETUP CHECKLIST

Telegram Setup

- [ ] Create bot with @BotFather
- [ ] Get bot token
- [ ] Get chat ID
- [ ] Install Node.js dependencies
- [ ] Create bot.js script
- [ ] Set environment variables
- [ ] Start bot
- [ ] Test commands

WhatsApp Setup

- [ ] Create Twilio account
- [ ] Get Account SID and Auth Token

- [ ] Enable WhatsApp Sandbox
- [ ] Install Node.js dependencies
- [ ] Create server.js script
- [ ] Setup ngrok tunnel
- [ ] Configure Twilio webhook
- [ ] Start server
- [ ] Test commands

TROUBLESHOOTING

Telegram Bot Not Responding

```
# Check if bot is running
ps aux | grep "node.*telegram-bot"

# View logs
tail -f ~/telegram-bot.log

# Restart bot
pkill -f "node.*telegram-bot"
nohup node ~/projects/aglc/telegram-bot/bot.js > ~/telegram-bot.log 2>&1 &

# Test bot token
curl https://api.telegram.org/bot<YOUR_TOKEN>/getMe
```

WhatsApp Bot Not Receiving Messages

```
# Check if server is running
ps aux | grep "node.*whatsapp-bot"

# Check ngrok tunnel
ngrok http 3000

# Verify Twilio webhook URL
# Go to Twilio Console and check webhook configuration

# View server logs
tail -f ~/whatsapp-bot.log
```

SECURITY NOTES

⚠ **Important:**

- Never share bot tokens or chat IDs
- Keep .env files private
- Use environment variables for credentials
- Restrict bot access to authorized users only
- Validate all incoming commands
- Log all command executions
- Use HTTPS for webhooks (ngrok provides this)

NEXT STEPS

1. **Choose Platform:** Telegram (easier) or WhatsApp (more users)
2. **Setup Bot:** Follow the steps above
3. **Test Commands:** Verify bot responds
4. **Setup Auto-Start:** Create systemd service

5. **Monitor:** Keep logs for troubleshooting

Status: READY FOR SETUP
Estimated Setup Time: 15-20 minutes
Support: Refer to troubleshooting section